

```
struct Edge {
    int to;
    double w;
};

bool check_shortest(int n, vector<vector<Edge>> &g) {
    vector<double> dis(n + 1, 0);
    vector<int> cnt(n + 1, 0);
    vector<int> inq(n + 1, 0);
    queue<int> q;

    // 判整个图有没有负环：所有点入队
    for (int i = 1; i <= n; i++) {
        q.push(i);
        inq[i] = 1;
    }

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        inq[u] = 0;

        for (auto [v, w] : g[u]) {
            if (dis[v] > dis[u] + w + EPS) {
                dis[v] = dis[u] + w;
                cnt[v]++;

                if (cnt[v] > n) {
                    return false; // 有负环，无解
                }

                if (!inq[v]) {
                    q.push(v);
                    inq[v] = 1;
                }
            }
        }
    }

    return true; // 无负环，有解
}

// 最长路差分约束:  $x[v] \geq x[u] + w$ 
bool spfa(int n, vector<vector<pair<int, double>>> &g) {
    vector<double> d(n + 1, 0);
    vector<int> cnt(n + 1, 0), inq(n + 1, 0);
    queue<int> q;

    for (int i = 0; i <= n; i++) {
        q.push(i);
        inq[i] = 1;
    }
}
```

```

while (!q.empty()) {
    int u = q.front();
    q.pop();
    inq[u] = 0;

    for (auto [v, w] : g[u]) {
        if (d[v] < d[u] + w - EPS) {
            d[v] = d[u] + w;
            cnt[v]++;

            if (cnt[v] > n + 1) return false;

            if (!inq[v]) {
                q.push(v);
                inq[v] = 1;
            }
        }
    }
}

return true;
}

```

约束	建边	用法
$x[v] \leq x[u] + w$	$u \rightarrow v, w$	最短路, 判负环
$x[v] \geq x[u] + w$	$u \rightarrow v, w$	最长路, 判正环
$x[v] - x[u] \leq w$	$u \rightarrow v, w$	最短路
$x[v] - x[u] \geq w$	$u \rightarrow v, w$	最长路
$x[v] == x[u] + w$	$u \rightarrow v, w$ 和 $v \rightarrow u, -w$	两条边